
EEE 6778. Applied Machine Learning II



Module 5. Transformers

Transformers Language Models

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a sense of depth and structure.

Self-Attention Recap

What is Self-Attention

Self-attention is the core mechanism of transformers that allows a model to focus on relevant parts of an input sequence when understanding or generating text.

Analogy: "Who Should I Listen To?"

Imagine you're in a classroom, trying to understand a complex topic. You have multiple classmates talking, but you don't listen to all of them equally:

- Some classmates give highly relevant points—*you focus on them.*
- Others talk off-topic—*you tune them out.*
- Some repeat what you already know—*you give them low priority.*

In self-attention, each word (token) in a sentence **listens** to every other word but decides **who is most relevant to focus on.**

What is Self-Attention

Let's say we have a sentence:

"The cat sat on the mat."

Each word is mapped to a vector representation:

$$X = [x_1, x_2, \dots, x_n]$$

Where x_i is a vector representing the word

Instead of treating each word independently, **self-attention allows each word to gather information from other words.**

Adding a Positional Encoding

Since self-attention alone does not capture position, we add positional encoding (PE):

$$X' = X + PE$$

where **PE** is either the sinusoidal encoding (fixed) or learnable embeddings (trainable).

The standard (non-learned) sinusoidal positional encoding used in "Attention Is All You Need" is defined as:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

where:

pos is the position index (e.g., 0, 1, 2, ... for each token),

i is the dimension index (ranging from 0 to $d/2$),

d is the embedding dimension.

Learning a PE

Some Transformer models use a learnable positional encoding (e.g., BERT). Instead of computing sinusoidal values, **they initialize a matrix of learnable parameters PE , which is updated during training.**

Feature	Sinusoidal PE (Fixed)	Learned PE (Trainable)
Generalization to longer sequences	✓ Yes (Extrapolates)	✗ No (Limited to trained lengths)
Adaptability to task	✗ No (Fixed, predefined)	✓ Yes (Learns task-specific positions)
Extra parameters?	✗ No (Fixed function)	✓ Yes (Requires trainable embeddings)
Computational cost	✓ Lower (No extra learning)	✗ Higher (More trainable parameters)
Usage in Transformers	✓ Used in Original Transformer (Vaswani et al.)	✓ Used in BERT, GPT, and some custom models
Relative Position Encoding?	✓ Yes (Encodes token distances naturally)	✗ No (Needs explicit design for relative positions)
Performance on seen sequence lengths	✗ Can be suboptimal	✓ Often performs better
Ease of Implementation	✓ Easy (Precomputed formula)	✓ Easy (Embedding layer)

Understanding Q, K, and V

Each token embedding X' in the sequence is projected into three different vectors:

1. **Query (Q):** "What am I looking for?"
2. **Key (K):** "What information do I have?"
3. **Value (V):** "What actual content should I share?"

These are learned linear transformations of the input:

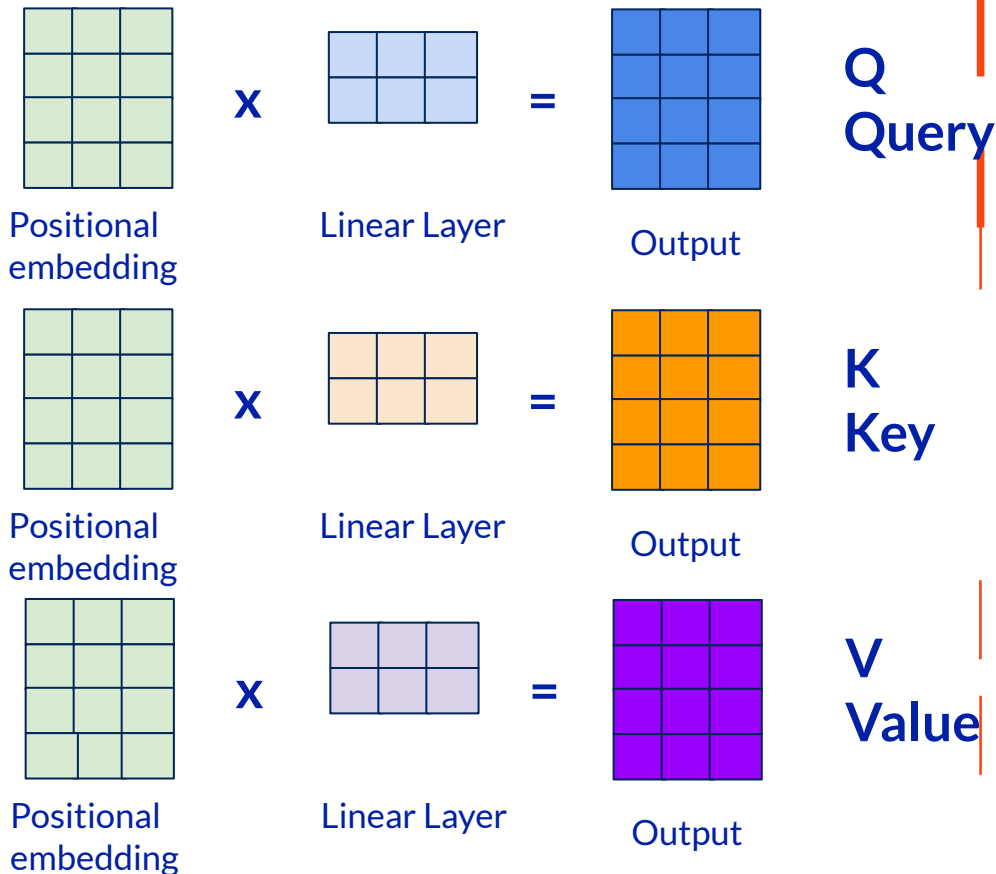
$$Q = X'W_Q, \quad K = X'W_K, \quad V = X'W_V$$

Where W_Q, W_K, W_V are learned weight matrices
(size embedding dimension x attention dimension)

Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search



Analogy: The Library Search

- **Query (Q)** → What kind of books are you looking for?
- **Key (K)** → Book descriptions in the library catalog.
- **Value (V)** → The actual book contents.

You match Query (your request) with Key (book descriptions) to find Value (relevant books).

Computing Attention Scores

Self-attention computes how much focus each token should have on every other token using the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\text{scaling}}\right) \cdot V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) \cdot V$$

d_k is the key dimension

We normalize the attention scores using softmax to ensure that: Attention scores are between 0 and 1. The sum of attention weights for each word is 1.

Weighted Sum of Values

Each token receives a new representation as a weighted sum of the Value vectors:

$$Z = \alpha V$$

- If "cat" is important to "The", its Value vector will be weighted **high**.
- If "television" is irrelevant, its Value vector will be weighted **low**

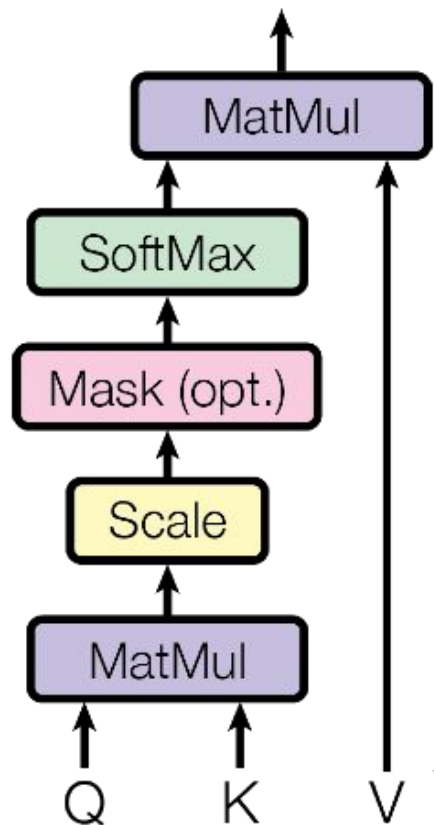
Weighted Sum of Values

Each token receives a new representation as a weighted sum of the Value vectors:

$$\text{New Representation} = \sum (\text{Attention Weight} \times V)$$

- If "cat" is important to "The", its Value vector will be weighted **high**.
- If "television" is irrelevant, its Value vector will be weighted **low**

Altogether - Self Attention Mechanism



Multi-Head Attention

The challenge with single self-attention is that it focuses on a single representation of relationships. In reality, different words may have different types of dependencies

To capture multiple perspectives, we use multiple heads:

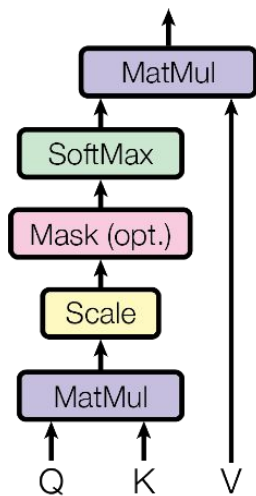
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

where:

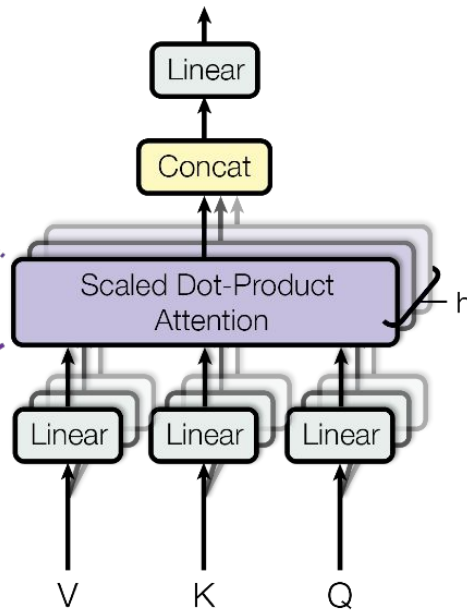
- Each head learns its own set of WQ, WK, WV
- Results from all heads are concatenated and projected using W_O

Altogether Multi-Head Attention

Scaled Dot-Product Attention



Multi-Head Attention



<https://medium.com/@pennQuin/attention-mechanisms-in-transformers-a726d8f5d5ee>

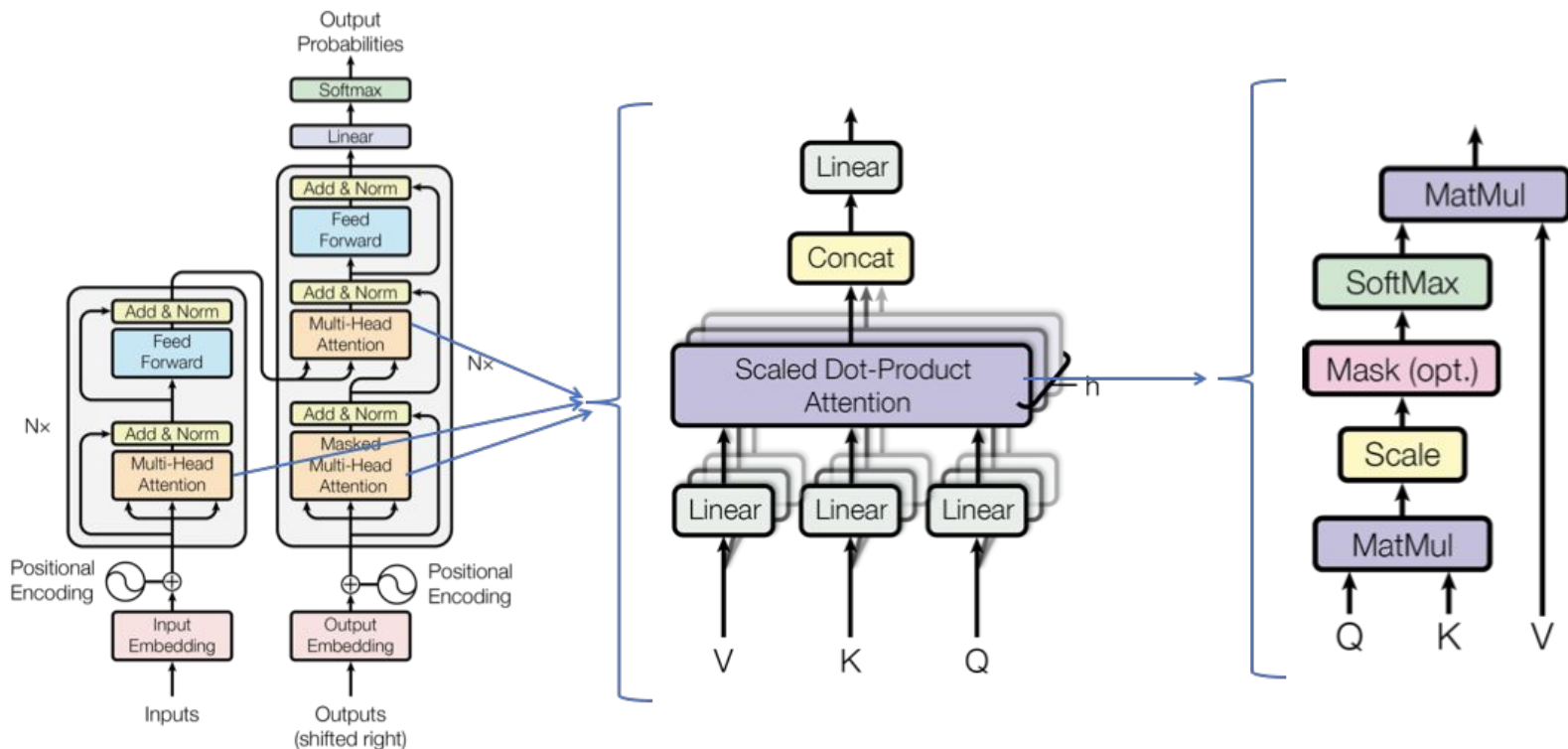


Extending to Transformers

From Multi-Head Attention to Transformers

- The Transformer model is a stack of layers, each containing
 - Multi-Head Self-Attention
 - Feedforward Network Layer
 - Normalization
 - Residual Connections

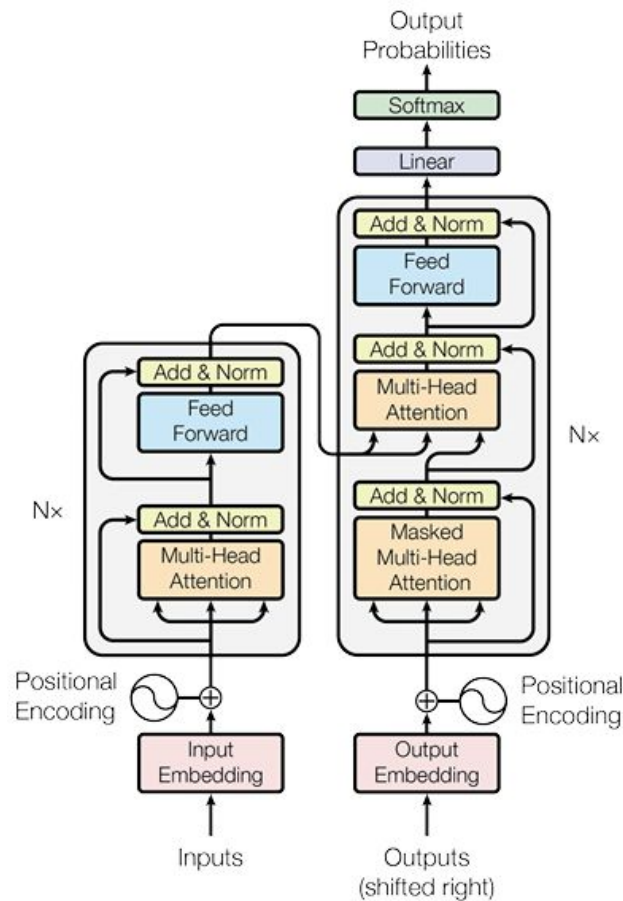
From Multi-Head Attention to Transformers



<https://medium.com/towards-data-science/self-attention-and-transformers-882e9de5edda>

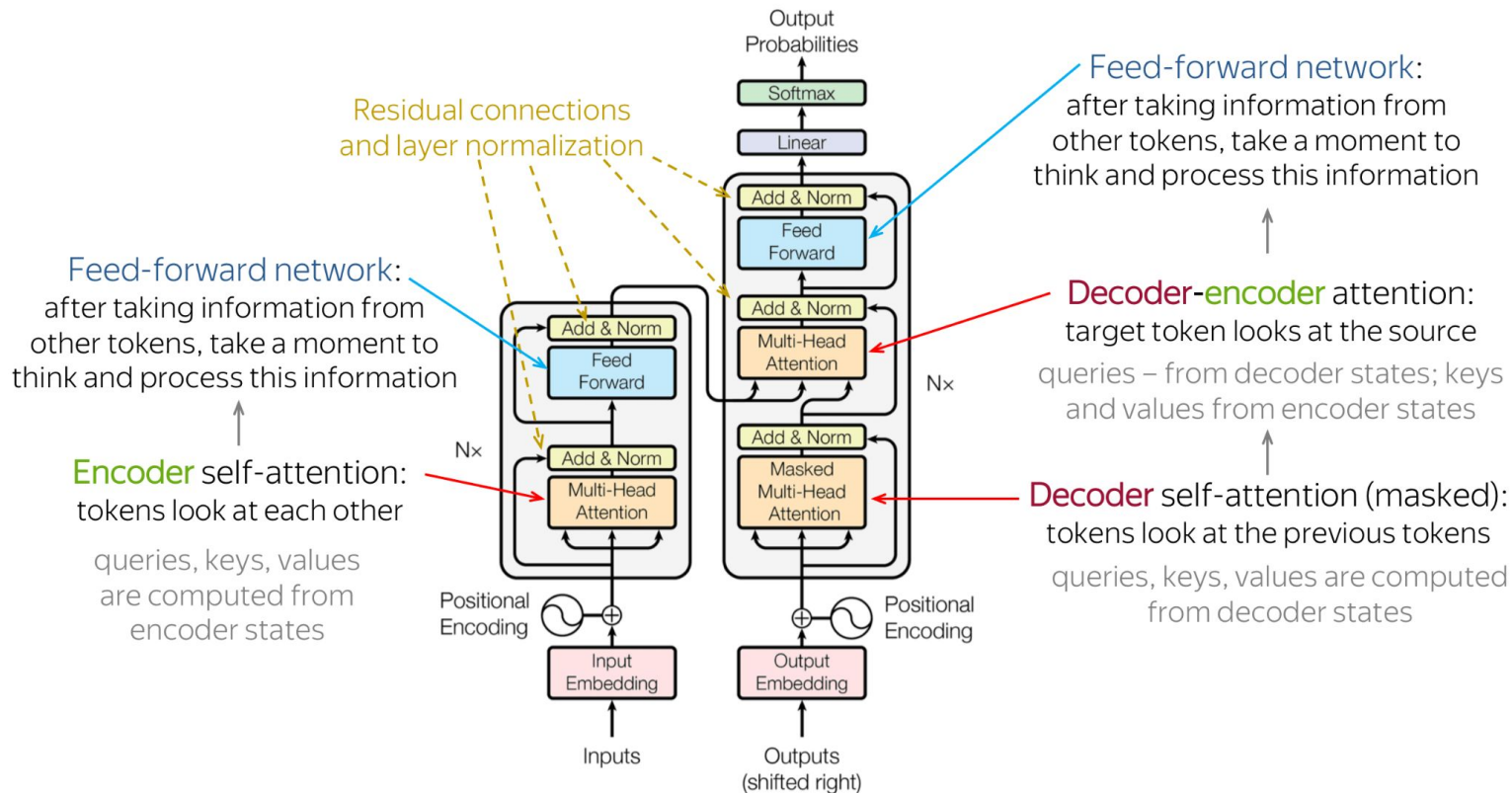
From Multi-Head Attention to Transformers

- Left (Encoder)
- Right (Decoder)
- Final Step



(Vaswani et al., 2017)

Vaswani et al., 2017 - Closer Look





Encoder Transformer - (Understanding)

BERT, RoBERTa, ALBERT → Used for classification, sentiment analysis, question answering.

Each encoder layer consists of:

- Multi-Head Self-Attention (each word attends to all words)
- Feedforward Network (FFN)
- Layer Normalization
- Residual Connections to stabilize training

Encoder Transformer - (Understanding)

Each encoder layer consists of:

- Multi-Head Self-Attention (each word attends to all words)

$$Z = \text{MultiHead}(X)$$

- Residual Connections + Layer Normalization

$$Z_{\text{norm}} = \text{LayerNorm}(X + Z)$$

- Feedforward Network (FFN)

$$F = \max(0, Z_{\text{norm}}W_1 + b_1)W_2 + b_2$$

- Final Layer Normalization (Output is an embedding)

$$Y = \text{LayerNorm}(Z_{\text{norm}} + F)$$

Decoder Transformer - (Generation)

GPT, LLaMA, Mistral → Used for text generation, chatbots, story writing.

The decoder is similar to the encoder but with two major differences:

- Masked Self-Attention
 - Prevents looking at future tokens to ensure autoregressive generation.
 - Uses a triangular mask so token t_i only sees previous tokens.
- Cross-Attention to Encoder
 - The decoder gets context from the encoder.
 - It uses encoder outputs as Keys and Values but keeps Queries from the decoder.

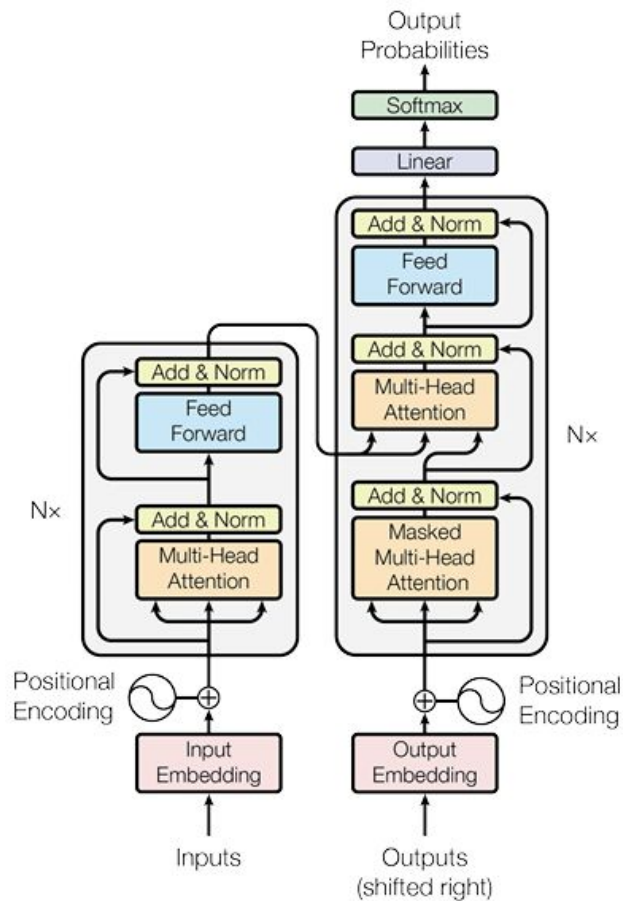
Instead of using self-attention, we now use the encoder outputs as context

$$\mathbf{A} = \frac{\mathbf{Q}_{\text{decoder}} \mathbf{K}_{\text{encoder}}^T}{\sqrt{d_k}}$$

$$\mathbf{Z} = \text{softmax}(\mathbf{A}) \mathbf{V}_{\text{encoder}}$$

Altogether - Transformers Architecture

- **Left (Encoder):** Processes input text by embedding words, adding positional information, and passing it through layers of multi-head attention (focusing on word relationships) and feed-forward networks (further refining meaning).
- **Right (Decoder):** Generates output text, using masked attention (to predict one word at a time) and cross-attention (to reference the encoded input).
- **Final Step:** A softmax layer selects the most probable next word.



(Vaswani et al., 2017)



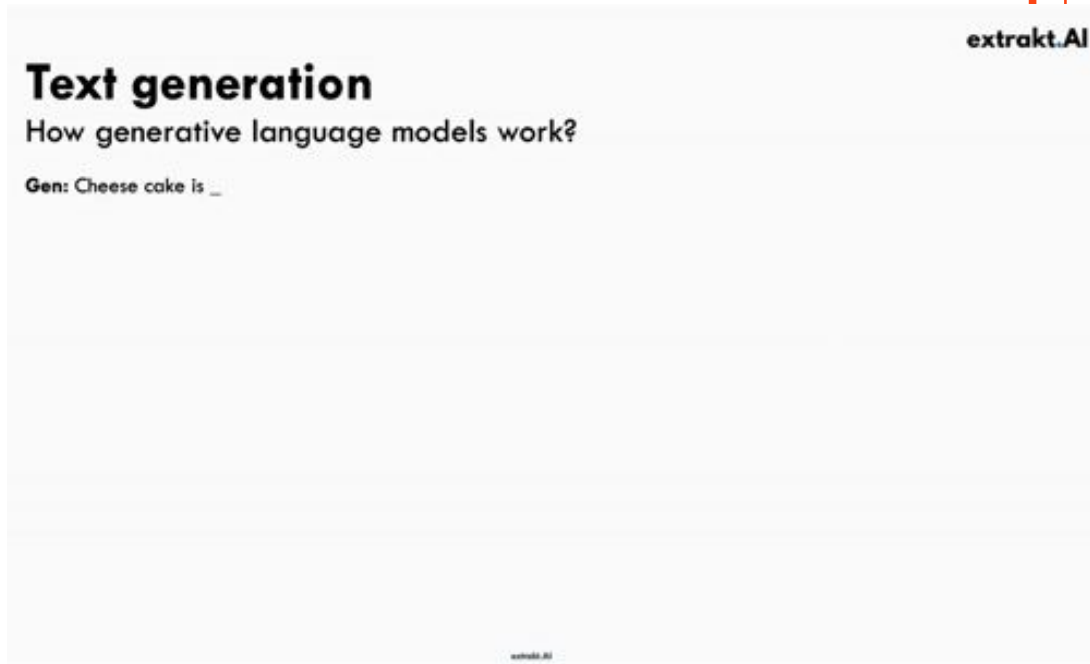
Large Language Models

Large Language Models

- LLMs are scaled-up versions of Transformer models trained on massive datasets, designed to generate, understand, and process text efficiently.
- **Key Features of LLMs:**
 - Trained on Billions/Trillions of Words
 - Uses Self-Supervised Learning (no labeled data needed)
 - Fine-Tuned for Many NLP Tasks (e.g., chatbots, text completion, summarization)
 - Massive Model Sizes (ranging from billions to trillions of parameters)

Text (and audio, image, etc.) Generation

- The Final Step in the Decoder Transform Architecture was: A softmax layer selects the most probable next word.
- LLM models generate content based on probabilities



Large Language Models

Model	Year	# Parameters	Notable Features
GPT-1	2018	117M	Transformer-based, OpenAI's first GPT
BERT (Large)	2018	340M	Larger bidirectional Transformer
GPT-2	2019	1.5B	Text generation, OpenAI withheld it initially
GPT-3	2020	175B	Few-shot, zero-shot learning breakthrough
GPT-4	2023	Unknown (~1T?)	Multi-modal capabilities
LLaMA 2 (Meta)	2023	7B, 13B, 70B	Improved LLaMA for commercial use
Claude 2 (Anthropic)	2023	Unknown (~100B?)	Trained for safety, context length 100k tokens
Gemini 1.5 (Google)	2024	Unknown (~1T?)	Large multi-modal model, 1M+ token context
o1-preview	2024	300B	Efficient reasoning over sheer size.

Large Language Models

- What make them so powerful?
 - **Parallelization!!**

Resources

- Understanding and Coding the Self-Attention Mechanism of Large Language Models From Scratch - [Sebastian Raschka](#)
- Transformer Explainer